

OXVERSE ACADEMY

Frontend Development Curriculum

Professional Diploma · Cohort-based · Project-driven

DURATION

3 Months (12 Weeks)

TOTAL WEEKS

12

LEVEL

Beginner → Intermediate → Advanced → Industry Ready

PROJECTS

15+

CAPSTONE

1 Enterprise-Level Project

PROGRAMME GOAL

Train students to become professional Frontend Engineers capable of working in startups, agencies, and enterprise teams.

OVERVIEW

A cohort-based, project-driven journey through the modern frontend stack. Students go from writing their first HTML tag to shipping a production React + TypeScript application with authentication, state management, testing, and deployment.

No 82, Century Bus Stop, Ago Palace Way, Okota, Lagos · +234 913 869 1147

Course Outline

1. **Week 1:** Professional Developer Environment
2. **Week 2:** HTML5 (Professional)
3. **Week 3:** CSS3 (Complete Professional)
4. **Week 4:** Git & GitHub
5. **Week 5:** Tailwind CSS
6. **Week 6:** JavaScript (Complete)
7. **Week 7:** TypeScript
8. **Week 8:** React.js (Professional)
9. **Week 9:** Professional Engineering Practices
10. **Week 10:** Career Accelerator
11. **Week 11:** E-commerce App with API
12. **Week 12:** Enterprise-Level Capstone Project

Professional Developer Environment

Week 1 establishes the mental model of the web and equips every student with a professional, reproducible development workstation and AI-augmented workflow before a single production line of code is written.

LEARNING OBJECTIVES

- Understand how the modern web works end-to-end, from DNS resolution to pixel rendering
- Set up a professional developer workstation with terminal, editor, and Node tooling
- Use AI coding assistants productively and responsibly inside a real workflow
- Navigate the command line and file system with confidence
- Understand the client-server model and how browsers negotiate connections

DETAILED TOPIC BREAKDOWN

1.1 Introduction to Software Development

What is Software Engineering? Frontend vs Backend vs Fullstack roles Web Architecture (client, server, database)

Client-Server Communication model Request Lifecycle (DNS → TCP → TLS → HTTP → render)

Browser Rendering Pipeline (parse, style, layout, paint, composite) Static vs Dynamic Websites

SPA vs MPA architectural tradeoffs Jamstack architecture CSR vs SSR vs SSG vs ISR

How job roles map to the stack (Junior/Mid/Senior Frontend)

1.2 Development Environment

VS Code Mastery (workspace, settings.json, multi-cursor) VS Code Shortcuts for speed editing

Essential Extensions (ESLint, Prettier, GitLens, Error Lens) Chrome DevTools (Elements, Console, Network, Sources, Lighthouse)

Node.js runtime and version management (nvm/fnm) npm package manager basics

pnpm workspaces and performance benefits yarn as an alternative package manager

Terminal Commands (cd, ls, mkdir, rm, mv, cp, grep, pipes) Environment Variables and .env files

Configuring a personal dotfiles setup

1.3 Internet Fundamentals

DNS resolution and record types (A, CNAME, MX) HTTP methods and status codes HTTPS and TLS handshake basics

IP Address and ports Browser Cache (memory cache, disk cache, cache headers)

Cookies (attributes, SameSite, Secure, HttpOnly) Sessions vs tokens Local Storage API Session Storage API

CORS (preflight requests, headers, common errors) CDN and edge delivery

1.4 Productivity & AI-Assisted Development

AI Coding Assistants overview ChatGPT for debugging and explanation GitHub Copilot inline suggestions and chat

Cursor as an AI-native editor Claude for large-context code review Gemini for multimodal coding tasks

Prompt Engineering for Developers (context, constraints, iteration) Verifying AI output and avoiding hallucinated APIs

Building a personal productivity system (Notion/Obsidian for dev notes)

1.5 Professional Habits

- Reading documentation effectively (MDN, official docs)
- Writing a clear bug report
- Time-boxing and the Pomodoro technique for coding
- Setting up a daily learning log
- Joining developer communities (Discord, Stack Overflow, X)

HANDS-ON EXERCISES

- Configure VS Code with 10 essential extensions and a custom settings.json
- Trace a full HTTP request in Chrome DevTools Network tab and annotate each phase
- Write 20 terminal commands to navigate and manage files
- Install and switch Node versions with nvm across two projects
- Diagram the browser rendering pipeline from memory

ASSIGNMENTS

- Document your development workflow in a README with screenshots
- Compare 3 AI coding assistants and write a short review with pros/cons
- Write a one-page explainer of CSR vs SSR vs SSG with real-world examples

PROJECTS

- Personal Dev Environment Setup Guide (Markdown)

WEEKLY LEARNING OUTCOMES

- Fully configured professional dev environment
- Confident with terminal, Git basics, and browser tooling
- Clear mental model of how the web works end-to-end
- Able to use AI tools as a productivity multiplier, not a crutch

WEEKLY ASSESSMENT

Live setup walkthrough + short quiz on web fundamentals

HTML5 (Professional)

A deep dive into writing semantic, accessible, and SEO-ready markup that forms a solid structural foundation for every subsequent styling and scripting layer.

LEARNING OBJECTIVES

- Write semantic, accessible, SEO-friendly HTML
- Build production-quality forms and multimedia experiences
- Diagnose and fix accessibility issues using automated and manual audits
- Structure documents for both machines (SEO/crawlers) and humans (screen readers)

DETAILED TOPIC BREAKDOWN

2.1 HTML Foundations

HTML Syntax and document structure Elements vs tags Attributes (global and element-specific) Comments
DOCTYPE and quirks mode Void elements Nesting rules Whitespace handling Character encoding (charset, entities)

2.2 Semantic HTML

header footer nav main section article aside figure/figcaption picture details/summary time
address mark when to use div vs semantic tags

2.3 Forms

Advanced form structure (fieldset, legend, label) Client-side Validation Pattern attribute and regex Accessible form labelling
Input Types (email, tel, date, range, color, file) Form APIs (Constraint Validation API) autocomplete and autofocus
Multi-step form UX

2.4 Multimedia

Video element and controls Audio element SVG inline vs external Canvas basics Responsive embeds (iframe)
Accessible captions and transcripts

2.5 SEO HTML

Meta Tags (title, description, viewport) Open Graph tags Twitter Cards Structured Data (JSON-LD)
Schema.org vocabularies Canonical URLs Sitemap and robots.txt basics

2.6 Accessibility

ARIA roles and attributes Screen Readers (VoiceOver/NVDA basics) Semantic Accessibility (landmark regions)
Keyboard Navigation and focus order Skip links Color contrast requirements Accessible naming (aria-label, aria-labelledby)

2.7 Performance

Lazy Loading images/iframes

Responsive Images

srcset and sizes attributes

picture element art direction

Preload/prefetch hints

Reducing DOM size for faster paint

HANDS-ON EXERCISES

- Rewrite a div-heavy page using semantic elements
- Build 5 form types with native validation and no JavaScript
- Add Open Graph + Schema.org metadata to a page
- Add ARIA landmarks and test with a screen reader
- Convert a fixed-size image gallery to a responsive srcset gallery

ASSIGNMENTS

- Run a Lighthouse accessibility audit and fix all issues
- Write a JSON-LD structured data block for a product page

PROJECTS

- Build a complete business website using semantic HTML only

WEEKLY LEARNING OUTCOMES

- Master of semantic, accessible, SEO-ready HTML5
- Able to build fully accessible forms without a framework
- Comfortable auditing and fixing accessibility violations

WEEKLY ASSESSMENT

Peer accessibility review + Lighthouse score ≥ 95

CSS3 (Complete Professional)

The complete modern CSS toolkit — from the cascade and box model to Grid, Flexbox, animation, and CSS architecture used in production design systems.

LEARNING OBJECTIVES

- Build responsive, animated, production-grade interfaces with modern CSS
- Master layout systems (Flexbox and Grid) for any UI pattern
- Apply a scalable CSS architecture across a multi-page project
- Optimize CSS for performance and maintainability

DETAILED TOPIC BREAKDOWN

3.1 CSS Fundamentals

Selectors (combinators, pseudo-classes, pseudo-elements)

Specificity calculation

Cascade and layers (@layer)

Inheritance

Box model (content-box vs border-box)

Units (px, em, rem, vh, vw, %)

3.2 Layout

Display types (block, inline, inline-block)

Position (static, relative, absolute, fixed, sticky)

Float and clearfix (legacy)

Flexbox (main/cross axis, grow/shrink/basis)

Grid (template areas, auto-fit/auto-fill, fr unit)

Multi-column layout

3.3 Typography

Font Loading strategies (font-display)

Variable Fonts

Responsive Typography with clamp()

Line-height and vertical rhythm

Web-safe fonts vs custom fonts

3.4 Responsive Design

Mobile First methodology

Breakpoints strategy

Media Queries (min-width, prefers-color-scheme)

Container Queries

Fluid layouts vs fixed breakpoints

3.5 Advanced CSS

Custom Properties (CSS Variables)

calc()

clamp()

min()

max()

CSS functions (var, attr)

3.6 Animation

Transition properties and easing

Keyframes

Timing Functions (cubic-bezier)

Transform (translate, scale, rotate)

3D Effects (perspective, transform-style)

will-change for performance

3.7 Modern CSS

Native Nesting

Logical Properties (inline/block)

Aspect Ratio

Object Fit / Object Position

Backdrop Filter

Mask and clip-path

:has() selector

3.8 CSS Architecture

BEM naming convention

Utility CSS philosophy

Component-scoped CSS

CSS Modules basics

Design tokens

3.9 CSS Optimization

Critical CSS extraction

Performance profiling with DevTools

Render Blocking CSS

Minimizing reflow/repaint

PurgeCSS concepts

HANDS-ON EXERCISES

- Recreate 5 hero sections from top SaaS sites
- Build a responsive grid gallery with hover animation
- Implement a sticky navbar with scroll-triggered style changes
- Build a pricing table using CSS Grid with equal-height cards

ASSIGNMENTS

- Publish a CSS pattern library with 15 reusable components

PROJECTS

- Clone Apple Homepage

WEEKLY LEARNING OUTCOMES

- Ability to translate any Figma design to pixel-perfect responsive CSS
- Confident with both Flexbox and Grid for any layout
- Can architect scalable CSS for a multi-page site

WEEKLY ASSESSMENT

Design-to-code challenge under time constraints

Git & GitHub

Professional version control workflows used in real engineering teams, from local branching to collaborative pull requests and open-source contribution.

LEARNING OBJECTIVES

- Collaborate professionally with Git, branches, and pull requests
- Resolve merge conflicts confidently
- Understand and apply common Git workflows used in teams
- Contribute to an external open-source codebase

DETAILED TOPIC BREAKDOWN

4.1 Git Core

- Git Installation and config (user.name/email)
- Repository initialization
- Branches and branch naming conventions
- Merge strategies (fast-forward, three-way)
- Rebase vs merge
- Cherry Pick
- Stash and stash pop
- Git log and reflog
- Undoing changes (revert, reset, checkout)

4.2 GitHub Workflow

- Pull Requests and code review etiquette
- Git Flow branching model
- Trunk-based development
- Collaboration via Issues and Projects
- Open Source Contribution etiquette
- Conflict Resolution strategies
- Protected branches and required reviews
- GitHub Actions basics for CI

HANDS-ON EXERCISES

- Simulate a merge conflict and resolve it
- Open a PR to an open-source project
- Practice interactive rebase to clean up commit history
- Recover a lost commit using reflog

ASSIGNMENTS

- Contribute one accepted PR to a public repository

PROJECTS

- Collaborative Team Project (3 devs, 1 repo)

WEEKLY LEARNING OUTCOMES

- Comfortable with branching strategies and code reviews
- Able to resolve merge conflicts without fear
- Understands team Git workflows used in industry

WEEKLY ASSESSMENT

PR review checklist scored by an instructor

Tailwind CSS

Utility-first CSS at production speed — building consistent, themeable design systems without leaving your markup.

LEARNING OBJECTIVES

- Build fast, consistent UIs with a utility-first CSS framework
- Configure and extend Tailwind's design tokens for a brand
- Compose reusable component patterns without custom CSS files
- Optimize Tailwind output for production

DETAILED TOPIC BREAKDOWN

5.1 Core Concepts

Installation (CLI, Vite plugin) Configuration file structure Theme Customization (colors, spacing, fonts)
Responsive Utilities (sm, md, lg, xl, 2xl) State variants (hover, focus, active, dark)

5.2 Layout

Flex utilities Grid utilities Container and max-width Gap utilities Positioning utilities

5.3 Styling

Typography utilities Color palette usage Spacing scale Shadows Borders and rings Opacity and blend modes

5.4 Components

Buttons with variants Cards Tables Forms Navigation bars Modals and dialogs

5.5 Customization

Tailwind Config extension Custom Utilities with @layer Plugins ecosystem Using CSS variables inside Tailwind theme

5.6 Optimization

Purging unused classes (content config) Tree Shaking in build Performance measurement
Using class-variance-authority for variants clsx/tailwind-merge patterns

HANDS-ON EXERCISES

- Convert a plain-CSS project to Tailwind
- Design a reusable button system with variants using cva
- Build a dark mode toggle using Tailwind's dark variant
- Build a responsive navbar with a mobile hamburger menu

ASSIGNMENTS

- Build a design system with 20 components in Tailwind

PROJECTS

- Modern SaaS Landing Page

WEEKLY LEARNING OUTCOMES

- Ship polished UIs 3× faster using Tailwind
- Comfortable extending Tailwind's theme for any brand
- Can build a component variant system without writing custom CSS

WEEKLY ASSESSMENT

Timed UI build challenge scored on fidelity and speed

JavaScript (Complete)

The largest module in the course. Deep, from-first-principles JavaScript covering language internals, asynchronous programming, the DOM, and browser APIs.

LEARNING OBJECTIVES

- Master modern JavaScript from language internals to browser APIs
- Understand execution context, closures, and the event loop deeply
- Write clean asynchronous code using promises and async/await
- Manipulate the DOM and consume browser APIs confidently

DETAILED TOPIC BREAKDOWN

6.1 Fundamentals

Variables (var/let/const) Scope (block, function, global) Hoisting Closures Functions (declarations, expressions, arrow)

Objects and property descriptors Arrays and array methods Loops (for, for...of, for...in, while)

Conditions and truthy/falsy values Type coercion

6.2 ES6+

Modules (import/export) Arrow Functions and lexical this Destructuring (object/array) Spread operator Rest parameters

Template Literals and tagged templates Optional chaining and nullish coalescing Symbols and iterators

6.3 Advanced JavaScript

Execution Context Call Stack Event Loop (microtasks vs macrotasks) Memory Management Garbage Collection

Prototype chain Prototypal Inheritance this Keyword resolution rules bind, call, apply Currying and partial application

6.4 Asynchronous JavaScript

Callback pattern and callback hell Promise (states, chaining, Promise.all/race/allSettled) Async Await syntax and error handling

Fetch API Axios library AbortController for cancellation Debounce and throttle patterns

6.5 DOM

Traversing the DOM tree DOM Manipulation (createElement, appendChild) Event Delegation Mutation Observer

Event bubbling vs capturing Custom events

6.6 Browser APIs

Local Storage API Session Storage API Geolocation API Clipboard API Notifications API Intersection Observer API

Resize Observer API Web Workers (introduction)

6.7 Error Handling

try/catch/finally

throw and custom Error classes

Debugging with breakpoints

Reading a stack trace

Defensive programming patterns

6.8 Design Patterns

Module Pattern

Factory Pattern

Singleton

Observer pattern

Pub/Sub pattern

HANDS-ON EXERCISES

- 30 daily JS katas
- Rebuild `Array.prototype.map/filter/reduce` from scratch
- Implement a debounce and throttle function from scratch
- Build a Promise polyfill
- Trace and diagram the event loop for a mixed sync/async script

ASSIGNMENTS

- Explain the event loop with your own diagram + example

PROJECTS

- Calculator
- Weather App
- Movie App
- Quiz App
- Expense Tracker

WEEKLY LEARNING OUTCOMES

- Deep, professional-level command of modern JavaScript
- Able to reason about async execution order confidently
- Comfortable working directly with the DOM and browser APIs

WEEKLY ASSESSMENT

Live coding interview covering closures, async, and the DOM

TypeScript

Adding static types to JavaScript to catch bugs earlier, document intent, and scale codebases safely across teams.

LEARNING OBJECTIVES

- Write safer, more maintainable JavaScript with types
- Model complex domain data with interfaces, generics, and unions
- Configure a strict TypeScript project from scratch
- Migrate an existing JavaScript codebase to TypeScript

DETAILED TOPIC BREAKDOWN

7.1 Type System

Type System basics (primitives, arrays, tuples) Interfaces vs type aliases Union and intersection Types Generics
Utility Types (Partial, Pick, Omit, Record) Type Narrowing with typeof/instanceof

7.2 Enums, Modules & Declarations

Enums vs const objects Modules and namespace resolution Declaration Files (.d.ts) Type Inference rules
Strict Mode flags (strictNullChecks, noImplicitAny) tsconfig.json deep dive

7.3 Advanced Types

Advanced Types (mapped types, conditional types) Type Guards and discriminated unions Template literal types
Generic constraints (extends keyof) Type-safe API client patterns

HANDS-ON EXERCISES

- Type 20 real-world function signatures
- Fix a broken generic API client
- Write a discriminated union to model API loading/success/error states
- Create 5 custom utility types from scratch

ASSIGNMENTS

- Publish a fully-typed utility npm package

PROJECTS

- Convert a JavaScript project into TypeScript

WEEKLY LEARNING OUTCOMES

- Confident authoring strict, well-typed TypeScript codebases
- Able to model complex domain logic with the type system
- Comfortable reading and writing declaration files

WEEKLY ASSESSMENT

Code review of a fully migrated TypeScript project

React.js (Professional)

The core production framework module — building full-featured React applications with routing, forms, authentication, state management, testing, and deployment.

LEARNING OBJECTIVES

- Ship production React applications with routing, state, auth, and testing
- Design reusable, composable component architectures
- Integrate real REST APIs with authentication and pagination
- Test and deploy a React application with confidence

DETAILED TOPIC BREAKDOWN

8.1 React Core

JSX syntax and rules Components (function components) Props and prop drilling State with useState
Component Lifecycle via useEffect Hooks rules Event Handling in React Conditional rendering Lists and keys

8.2 Advanced React

Custom Hooks Compound Components pattern Context API Performance Optimization (useMemo, useCallback)
Memoization with React.memo Code Splitting Lazy Loading with React.lazy/Suspense useReducer for complex state

8.3 Routing

React Router setup Nested Routes Protected Routes Dynamic Routes and params Route loaders and data fetching
404/error routes

8.4 Forms

React Hook Form setup Schema Validation with Zod Controlled vs uncontrolled Components Field arrays and dynamic forms
Form error UX

8.5 State Management

Context API for global state Redux Toolkit (slices, thunks) Zustand (Introduction) Server State vs Client State
TanStack Query for server state

8.6 API Integration

Axios instance configuration REST API consumption patterns Authentication headers Pagination strategies Infinite Scroll
Optimistic Updates Handling loading/error states

8.7 Authentication

JWT structure and storage strategies Refresh Tokens flow Protected Pages/route guards
Role-Based Access Control (Frontend) Persisting auth state

8.8 Advanced Topics

Error Boundaries Suspense for data fetching Portals Higher-Order Components Render Props pattern

Composition Patterns over inheritance

8.9 Testing

Unit Testing with Vitest/Jest React Testing Library queries Component Testing best practices Mocking API calls in tests

Snapshot testing tradeoffs

8.10 Deployment

Vercel deployment workflow Netlify deployment workflow Environment Variables in production

Build Optimization (bundle analysis) CI checks before deploy

HANDS-ON EXERCISES

- Rewrite 3 vanilla JS apps as React apps
- Build 5 custom hooks (useDebounce, useFetch, useLocalStorage, etc.)
- Build a protected route wrapper with role checks
- Write RTL tests for a form component

ASSIGNMENTS

- Implement JWT auth flow with refresh tokens

PROJECTS

- Enterprise Admin Dashboard

WEEKLY LEARNING OUTCOMES

- Ready to work on production React codebases in any team
- Comfortable choosing the right state management tool for a problem
- Able to test and deploy a React app end-to-end

WEEKLY ASSESSMENT

React coding challenge + component design review

Professional Engineering Practices

Shifting from writing code that works to writing code that scales — clean architecture, reusable systems, and rigorous audits used in senior engineering roles.

LEARNING OBJECTIVES

- Work like a senior engineer: clean code, architecture, audits
- Apply SOLID-inspired principles to frontend component design
- Structure a scalable component and folder architecture
- Run and act on accessibility, performance, and security audits

DETAILED TOPIC BREAKDOWN

9.1 Code Quality

Clean Code principles SOLID Principles (Frontend Perspective) Folder Structures (feature-based vs type-based)
Naming conventions Code review checklists ESLint/Prettier enforcement

9.2 Architecture

Component Architecture layering Reusable UI Systems Atomic Design (atoms, molecules, organisms)
Design tokens and theming architecture Monorepo basics for shared UI

9.3 Audits

Accessibility Audits (axe, WAVE) Lighthouse Audits (performance, SEO, best practices)
Performance Optimization (bundle size, Core Web Vitals) Security Best Practices (XSS, CSP, dependency audits)
SEO for React Applications (meta management, SSR considerations)

9.4 Documentation & Collaboration

Writing architecture decision records (ADRs) Storybook for component documentation Onboarding docs for new engineers
Cross-functional collaboration with design/PM

HANDS-ON EXERCISES

- Refactor a messy component into an atomic design system
- Run and fix a full Lighthouse audit
- Write an ADR for a state management decision
- Set up Storybook for 5 shared components

ASSIGNMENTS

- Write an architecture doc for your capstone

PROJECTS

- Design system + audit report for a real product

WEEKLY LEARNING OUTCOMES

- Think and communicate like a senior frontend engineer

- Able to structure large codebases for maintainability
- Comfortable running and interpreting technical audits

WEEKLY ASSESSMENT

Architecture review presentation to peers

Career Accelerator

Turning technical skill into a job offer — portfolio, personal brand, interview preparation, and freelance income channels.

LEARNING OBJECTIVES

- Land your first (or next) frontend role
- Build a portfolio that demonstrates real engineering judgment
- Prepare for technical and behavioral interviews
- Establish freelance or contract income channels

DETAILED TOPIC BREAKDOWN

10.1 Portfolio

GitHub Portfolio curation Technical Resume writing LinkedIn Optimization Writing compelling case studies
Pinning and presenting top repos

10.2 Work Channels

Freelancing (Upwork, Fiverr, Contra) Cold outreach to startups Building a referral network Pricing your first freelance projects

10.3 Interviews

Technical Interviews structure Whiteboard & Live Coding Preparation System design basics for frontend
Behavioral interview frameworks (STAR) Salary negotiation basics

10.4 Brand

Personal Branding strategy Building a Developer Portfolio site AI-Assisted Development Workflows for productivity
Writing technical blog posts Building in public on X/LinkedIn

HANDS-ON EXERCISES

- Mock interview with feedback
- Rewrite your LinkedIn headline & about section
- Do 10 live-coding practice problems under time pressure
- Draft 3 outreach messages for freelance leads

ASSIGNMENTS

- Publish a portfolio site + 5 case studies

PROJECTS

- Personal Portfolio Website

WEEKLY LEARNING OUTCOMES

- Interview-ready resume, LinkedIn, and portfolio
- Comfortable navigating technical and behavioral interviews
- Has an active freelance or job-search pipeline

WEEKLY ASSESSMENT

Mock interview scored by an instructor panel

E-commerce App with API

A full production-shaped e-commerce build integrating real APIs, cart persistence, checkout, and authenticated account areas — the flagship pre-capstone project.

LEARNING OBJECTIVES

- Ship a real, deployed e-commerce experience
- Integrate a real payment provider in test mode
- Manage complex global state across cart, auth, and orders
- Handle real-world edge cases (out of stock, failed payment, empty states)

DETAILED TOPIC BREAKDOWN

11.1 Product & Cart

Product listing with filtering and sorting Product detail pages Filters and search Cart state management
Persistent cart (localStorage/backend sync) Quantity and stock validation

11.2 Checkout

Address form with validation Payment integration (Stripe test mode) Order confirmation and receipts Coupon/discount logic
Shipping calculation basics

11.3 Auth & Account

Sign up / sign in flows Order history page Protected account routes Wishlist/favorites feature Profile editing

11.4 Polish & Edge Cases

Empty and error states Loading skeletons Toast notifications Responsive checkout UX Analytics events on key actions

HANDS-ON EXERCISES

- Wire cart to global state with optimistic updates
- Build a coupon code validation flow
- Add skeleton loaders for product listing

ASSIGNMENTS

- Integrate a real payment provider in test mode

PROJECTS

- Deployed E-commerce App with API

WEEKLY LEARNING OUTCOMES

- Portfolio-grade e-commerce project
- Real experience with payment integration
- Confident handling complex, multi-page application state

WEEKLY ASSESSMENT

Full walkthrough demo covering cart, checkout, and account flows

Enterprise-Level Capstone Project

The final, portfolio-defining project: a fully planned, built, tested, and deployed enterprise-grade application presented to a panel of hiring partners.

LEARNING OBJECTIVES

- Deliver a production-ready, deployed capstone
- Practice full-cycle product planning from spec to deployment
- Present technical decisions clearly to a non-technical and technical audience
- Ship with proper CI/CD, monitoring, and documentation

DETAILED TOPIC BREAKDOWN

12.1 Planning

Writing a product spec System architecture diagram Component tree planning API contract design
Choosing the tech stack and justifying tradeoffs

12.2 Build

React + TypeScript + Tailwind implementation Integrating real APIs Authentication and authorization
State management at scale Responsive design across breakpoints Accessibility pass

12.3 Ship

CI/CD pipeline setup Deployment to production Analytics integration Error tracking (Sentry)
Writing full technical documentation Post-launch monitoring plan

12.4 Presentation

Structuring a demo day pitch Live-demoing under pressure Answering technical Q&A Presenting metrics and impact

HANDS-ON EXERCISES

- Weekly demo + peer code review

ASSIGNMENTS

- Write full technical documentation and README

PROJECTS

- Enterprise Capstone (deployed) with polished GitHub repo

WEEKLY LEARNING OUTCOMES

- A portfolio-defining, production-ready project
- Full-cycle experience from spec to shipped, monitored product
- Confidence presenting technical work to hiring panels

WEEKLY ASSESSMENT

Panel demo day with hiring partners